CUPS Printer Discovery — Exploitation and Attack Detection Using Scapy

DAT505 — Ethical Hacking, University of Stavanger

Bjarte Landråk

16 November 2026

Contents

1. Introduction **2**

2. Task 1 — Virtual Environment Setup **2**
2.1 Topology . . . 2
2.2 Victim deployment . . . 3
2.3 Baseline capture . . . 3

3. Task 2 — CUPS Attack Tool (`cups_attack.py`) **3**
3.1 Design . . . 3
3.2 Usage . . . 3

4. Task 3 — Detection Tool (`cups_detector.py`) **4**

5. Task 4 — Attack Simulation and Analysis **4**
5.1 Experiment 1 — Normal traffic . . . 4
5.2 Experiment 2 — Unauthorized discovery . . . 4
5.3 Experiment 3 — Packet manipulation (three variations) . . . 4
5.4 Experiment 4 — Detection results . . . 5
5.5 Evaluation — detections, false positives, missed attacks . . . 5

6. Task 5 — Defence and Mitigation **5**

7. Ethical Considerations **6**

8. Conclusion **6**

9. References **6**

1. Introduction

The Common Unix Printing System (CUPS) is the default printing stack on most Linux distributions. In September 2024 a chain of vulnerabilities was disclosed in the CUPS ecosystem — most importantly in the `cups-browsed` service — that allows an unauthenticated attacker on the network to have a malicious printer automatically registered on a victim system, which can escalate to remote command execution when a print job is sent:

CVE	Component	Issue
CVE-2024-47176	`cups-browsed`	Binds to `0.0.0.0:631/udp` and trusts unauthenticated `Get-Printer-Attributes` IPP requests triggered by incoming browse packets.
CVE-2024-47076	`libcupsfilters`	`cfGetPrinterAttributes` does not validate IPP attributes returned by the advertised printer.
CVE-2024-47175	`libppd`	`ppdCreatePPDFromIPP2` writes attacker-controlled IPP data into a PPD file without sanitisation.

This report investigates the **insecure printer-discovery mechanism** at the root of the chain (CVE-2024-47176). Two Scapy-based tools were developed: `cups_attack.py`, which advertises an attacker-controlled virtual printer to a vulnerable host, and `cups_detector.py`, which detects suspicious CUPS discovery activity. In line with the assignment, full remote code execution was **not** the objective — the focus is on demonstrating and detecting the trust failure in printer discovery.

2. Task 1 — Virtual Environment Setup

2.1 Topology

An isolated host-only laboratory was used so that no traffic could leave the lab network.

Role	Host	Address	Software
Attacker	Kali/Ubuntu	`192.168.56.10`	Python 3, Scapy, Wireshark/tcpdump
Victim	Docker (Vulhub)	`192.168.56.20`	`cups-browsed` (CVE-2024-47177 dir), UDP/631
Virtual printer	`ippserver`	`192.168.56.20:631`	OpenPrinting IPP server

No physical printer was required; OpenPrinting's `ippserver` provided a software IPP endpoint, and the vulnerable CUPS service was supplied by the Vulhub project.

2.2 Victim deployment

```
git clone https://github.com/vulhub/vulhub
cd vulhub/cups-browsed/CVE-2024-47177
docker compose up -d        # exposes cups-browsed on UDP/631
```

2.3 Baseline capture

Normal discovery traffic between CUPS and the virtual printer was captured with `tcpdump -i eth0 udp port 631 -w pcap_files/normal.pcap`. A legacy CUPS browse advertisement is a single UDP/631 text line of the form:

```
<type-hex> <state-hex> ipp://<host>:<port>/printers/<name> "loc" "info" "make-model"
```

The captured baseline (see `evidence/pcap_readout_normal.txt`) shows the printer `192.168.56.20` advertising itself on port 631 — the expected, legitimate case.

3. Task 2 — CUPS Attack Tool (`cups_attack.py`)

3.1 Design

`cups_attack.py` uses Scapy to craft `IP/UDP/Raw` packets carrying a legacy CUPS browse advertisement and sends them to the victim's UDP/631. By controlling the advertised `ipp://` URI it advertises a **virtual printer under the attacker's control**, demonstrating that `cups-browsed` acts on unauthenticated input.

Implemented features (all assignment requirements):

- **Custom discovery packets** built with Scapy (`IP()/UDP()/Raw()`).
- **Attacker-controlled printer** via `--listener-ip` / `--listener-port` / `--name`, producing the advertised URI.
- **Field manipulation** of printer address, port, type/state and make-and-model, including an optional spoofed source IP (`--src-ip`).
- **Victim-interaction demonstration**: when run live (not `--dry-run`) against the Vulhub container, `cups-browsed` connects back to the advertised URI, observable as an inbound IPP `Get-Printer-Attributes` to the attacker's `ippserver` — proving the victim trusts the advertisement.
- **CLI, logging and PCAP export** (`argparse`, Python `logging`, `wrpcap`).

A `--dry-run` mode crafts and saves packets without transmitting, which makes the captures in this report fully reproducible and safe.

3.2 Usage

```
# Live rogue advertisement against the victim
sudo python3 cups_attack.py -t 192.168.56.20 \
    --src-ip 192.168.56.10 --listener-ip 192.168.56.10 \
    --listener-port 8631 -n EvilPrinter -m "Rogue IPP Everywhere"

# Three crafted variations, exported to a PCAP
python3 cups_attack.py -t 192.168.56.20 --src-ip 192.168.56.10 \
```

```
--listener-ip 192.168.56.10 --variations -c 3 --dry-run \
--pcap pcap_files/attack_variations.pcap
```

4. Task 3 — Detection Tool (`cups_detector.py`)

`cups_detector.py` is a defensive Scapy tool that inspects CUPS discovery traffic either live (`-l <iface>`, via `sniff`) or offline (`-r <pcap>`, via `rdpcap`). It implements the four required detections:

1. **Unauthorized printer advertisement** — a browse packet whose source IP is not on the configured allowlist (`--allow`).
2. **Suspicious IPP connection / URI** — an advertised `ipp://` URI whose host is outside the allowed ranges, or whose port is not in `--allowed-ports`.
3. **Repeated / abnormal discovery rate** — more than `--rate-threshold` advertisements from one source within a sliding `--window`.
4. **Malformed / unexpected protocol fields** — a line that fails to parse, has an out-of-range type/state value, or oversized fields.

Alerts carry a timestamp, source, category, severity and detail, and are exported to **CSV or JSON** (`--format`). The allowlist model makes the tool directly reusable as a lightweight network IDS signature for CUPS.

5. Task 4 — Attack Simulation and Analysis

Four experiments were performed. The committed PCAPs, logs and alert exports are in `pcap_files/` and `evidence/`.

5.1 Experiment 1 — Normal traffic

The legitimate printer advertises itself (`normal.pcap`). Running the detector with the printer on the allowlist produced **0 alerts** — no false positives:

```
Analysing 3 packet(s) from pcap_files/normal.pcap
Exported 0 alert(s) to evidence/alerts_normal.csv
Total alerts: 0
```

5.2 Experiment 2 — Unauthorized discovery

The attack tool advertises a rogue printer from `192.168.56.10` pointing at the attacker's IPP endpoint. The detector immediately raised **HIGH** severity `unauthorized-advertisement` and `suspicious-ipp-uri` alerts.

5.3 Experiment 3 — Packet manipulation (three variations)

Three crafted variations were tested (labels as used by the attack tool):

Variation 1 — baseline. A straightforward rogue advertisement. Detector response: unauthorized-advertisement and suspicious-ipp-uri.

Variation 2 — alt-port / spoofed model. The advertised printer uses an alternate port (9100), a spoofed trusted make-and-model ("HP Color LaserJet MFP") and an altered state field. Detector response: the above plus suspicious-ipp-port.

Variation 3 — malformed fields. Out-of-range protocol fields (type = 0xffffffff, state = 0x7f) and a 200-byte oversized field. Detector response: the above plus abnormal-protocol-fields and oversized-field.

5.4 Experiment 4 — Detection results

Running the detector over `attack_variations.pcap` (9 packets) produced **37 alerts** spanning all four detection categories, including `abnormal-discovery-rate` once the sliding window exceeded five advertisements:

```
[ALERT][HIGH]   unauthorized-advertisement :: ipp://192.168.56.10:8631/printers/EvilPrinter
[ALERT][HIGH]   suspicious-ipp-uri         :: URI points to non-allowlisted host 192.168.56.10
[ALERT][MEDIUM] suspicious-ipp-port        :: advertised printer uses unusual port 8631
[ALERT][MEDIUM] abnormal-discovery-rate    :: 6 advertisements in 10.0s window
[ALERT][MEDIUM] abnormal-protocol-fields   :: type=0xffffffff state=0x7f out of normal range
[ALERT][LOW]    oversized-field            :: browse field exceeds expected length
Total alerts: 37
```

5.5 Evaluation — detections, false positives, missed attacks

- **Successful detections:** every crafted advertisement was flagged. The URI-based and field-based checks caught manipulation that source-IP filtering alone would miss.
- **False positives:** none on normal traffic (0 alerts), because the legitimate printer is allowlisted and uses port 631.
- **Missed attacks / limitations:** if an attacker is placed *inside* the allowlisted subnet and advertises a URI that is also inside it on port 631, source- and URI-based checks would not fire; detection then relies on the rate and malformed-field heuristics, or on anomaly detection of *new* printer names. This is an inherent limit of allowlist-based detection and is noted as future work (e.g. tracking first-seen printer URIs).

6. Task 5 — Defence and Mitigation

The mitigation applied was **restricting UDP/631** so that only the trusted printer subnet can reach `cups-browsed`:

```
sudo iptables -A INPUT -p udp --dport 631 -s 192.168.56.20 -j ACCEPT
sudo iptables -A INPUT -p udp --dport 631 -j DROP
# or, where discovery is not needed:
sudo systemctl disable --now cups-browsed
```

Before vs. after:

Stage	Advertisement reaches `cups-browsed`?	Detector alerts
Before	Yes — parsed and acted upon	37
After	No — dropped at the firewall	0

After the rule was applied, the attacker's packets no longer reached the service, no rogue printer was auto-created, and the detector observed no attacker advertisements (`evidence/mitigation.md`).

Additional recommended controls: disabling unnecessary `cups-browsed` functionality, allowlisting authorised printer addresses, applying vendor patches, and network segmentation of printers.

7. Ethical Considerations

All work was carried out in an isolated, host-only laboratory against systems owned by the author (a Vulhub Docker container and a local `ippserver`). No third-party or production systems were touched, and the attack tool is limited to the discovery mechanism — it contains no remote-code-execution payload. The `--dry-run` mode was used to craft reproducible captures without transmitting packets. These vulnerabilities are publicly documented (NVD, OpenPrinting advisories); the tools are intended for defensive research and education, and should only be used with explicit authorisation. The accompanying detection tool and mitigation guidance are provided to help defenders, consistent with responsible-disclosure norms.

8. Conclusion

The experiments confirmed the core weakness behind the 2024 CUPS chain: `cups-browsed` accepts and acts on unauthenticated printer advertisements. The Scapy attack tool reliably advertised an attacker-controlled printer, the three crafted variations exercised address, port and protocol-field manipulation, and the Scapy detector flagged all of them with zero false positives on normal traffic. Restricting UDP/631 eliminated the attack surface, demonstrating an effective, low-cost mitigation.

9. References

1. OpenPrinting / NVD — CVE-2024-47176, CVE-2024-47076, CVE-2024-47175 (2024).
2. Vulhub — `cups-browsed/CVE-2024-47177` vulnerable environment. https://github.com/vulhub/vulhub/tree/master/cups-browsed/CVE-2024-47177
3. Scapy documentation. https://scapy.readthedocs.io/
4. OpenPrinting `ippserver` / IPP Everywhere specification.